M7 ESP32 Software and Communication — Module Proposal

Nov 1, 2026 · @Taha

1. Summary and scope

M7 is the firmware on the ESP32 inside the LED board and the protocol between the board and the main computer. It receives the game state over Wi-Fi, hands it to the LED engine (M8), reads the motion sensor, detects hits and estimates their strength and area, drives the vibration motors, and reports status and sensor data back. The board protocol defined here is the only way any module talks to the board.

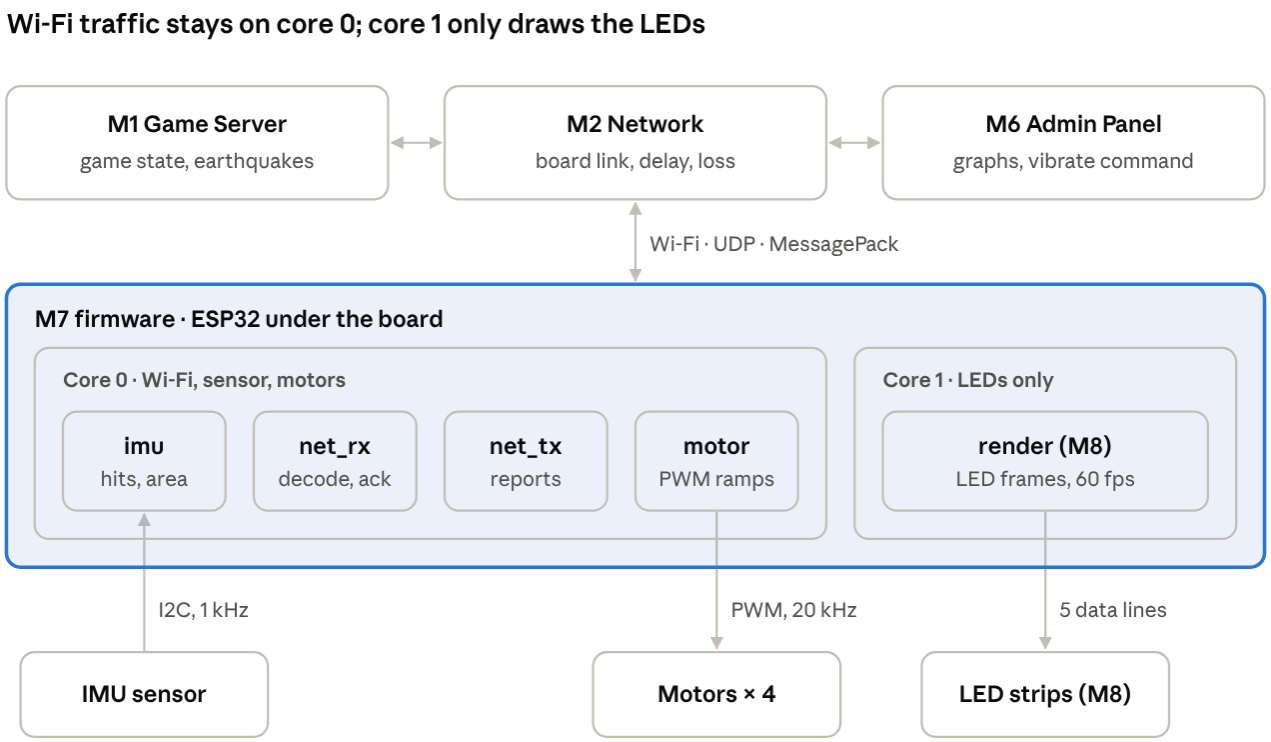
Item	Detail
Members	Tuğberk Akbulut, Hasan Alperen Elmas, Hasan Hüseyin Özdemir, Muhib Taha Boy
Runs on	ESP32 microcontroller under the board, plus two small PC test tools
In scope	Wi-Fi link and board protocol, connection recovery, IMU driver, hit detection and localisation, vibration motor control, status and latency reports, over-the-air updates
Out of scope	Light animations and LED output (M8), applying admin network delay and loss (M2), turning a hit into game effects (M1)

Three design rules keep M7 stable and testable:

- **State, not commands.** The server sends the full visible board state 30 times per second. A lost packet is simply replaced by the next one, so the board never gets out of sync.
- **One protocol, three implementations, one set of test vectors.** The same message examples are checked by the ESP32 code, the server code and the PC test tools.
- **The board runs alone.** Without a server it shows a demo or an idle animation, and it reconnects by itself.

2. Position in the system

M7 is the only bridge between the main computer and the physical board. M1 and M6 reach the board through M2's board link; on the ESP32, M7's tasks on core 0 feed the M8 LED engine on core 1 and drive the sensor and motors.



M7 in the system · server side, two ESP32 cores, board hardware

The network is crossed by one protocol (Section 4) and the two cores by one C++ interface (Section 3.2). Each side of both lines has an emulator, so all four neighbours can be tested without M7 hardware.

3. Interface contract: inputs and outputs

M7 has three interfaces: the network link to the main computer, a C++ interface to the LED engine (M8) on the same chip, and the hardware pins. All three are fixed here.

3.1 Network side (main computer)

Direction	Message	Rate	Reliability	Used by
In	`STATE`: players, delivery points, obstacles, earthquake, game phase	30 Hz (10–60 configurable)	None needed; next packet replaces it	M1 via M2
In	`CONFIG`: team colours, brightness limit, animation settings	At connect and on change	Acknowledged, resent	M1, M6
In	`VIBRATE`: motor mask, strength, duration	On admin command	Acknowledged, resent	M6 via M1
In	`PING`: server time	2 Hz	None	M2
Out	`HELLO`: firmware and city plan version, board ID	Every 1 s until answered	Repeated	M2
Out	`QUAKE`: source, strength, estimated area, confidence	On a detected hit	Acknowledged, resent	M1, M6
Out	`SENSOR`: acceleration samples for the seismic graph	10 Hz, 10 samples each (100 Hz)	None	M6
Out	`STATUS`: last applied state, frame rate, Wi-Fi signal, power estimate, errors	1 Hz	None	M6
Out	`PONG`: echo of ping, time the last state was shown	2 Hz	None	M2, M6 (board delay graph)

3.2 Firmware side (to M8, same ESP32)

```
// M7 calls these; M8 implements them. No Wi-Fi or Arduino types cross this
line.
void ledEngine_applyConfig(const BoardConfig& cfg);   // colours, brightness,
power limit
void ledEngine_applyState(const BoardState& st);      // latest state, copied
in a double buffer
void ledEngine_setLinkStatus(LinkStatus s);           // CONNECTED,
SEARCHING, LOST
uint32_t ledEngine_lastFrameMicros();                 // M7 reports this as
board delay
```

`BoardState` and `BoardConfig` are plain C++ structs that mirror the `STATE` and `CONFIG` messages field by field (Section 4). M8 can fill them from a demo script without Wi-Fi, and M7 can test with an empty LED engine that only logs.

3.3 Hardware side

Signal	Part	ESP32 pins (proposed)
I²C at 400 kHz	Motion sensor (IMU)	GPIO 21 (SDA), 22 (SCL), 19 (interrupt)
PWM at 20 kHz	4 vibration motor drivers	GPIO 25, 26, 27, 33
LED data	Owned by M8	GPIO 4, 16, 17, 18, 23
Spare	Status LED, push button for demo mode	GPIO 2, 0

The pins avoid the ESP32 strapping pins that change boot behaviour (except GPIO 0 and 2, which are safe for a button and an LED) and the input-only pins 34–39.

4. Communication protocol and message formats

The board protocol is UDP on the local Wi-Fi with MessagePack-encoded messages. The main computer listens on port 47100 and the board on port 47101. Section 5 compares this with WebSocket, MQTT and a hand-made binary format.

4.1 Envelope

Every message has the same outer fields. Examples are shown as JSON; on the wire the same structure is MessagePack, which is about half the size and is read by the same ArduinoJson library on the ESP32.

Field	Type	Meaning
`t`	1 character	Message type: `S` state, `C` config, `V` vibrate, `P` ping, `H` hello, `Q` quake, `N` sensor, `T` status, `O` pong, `A` ack
`v`	integer	Protocol version, starts at 1; the board rejects a different version and reports it in `STATUS`
`seq`	integer	Sequence number per sender; older or repeated messages are ignored
`ts`	integer	Sender time in ms (server: game time; board: time since boot)
`ack`	true / absent	If true, the receiver must answer with `A` and the same `seq`
`d`	object	Payload

4.2 STATE (server → board, 30 Hz)

Lists are arrays instead of objects to keep the packet small. Positions use the normalised city coordinates `u, v` from the shared city plan (0 = north-west corner intersection, 1 = south-east).

```
{ "t": "S", "v": 1, "seq": 52031, "ts": 60800,
  "d": {
    "ph": 2, "tl": 239,
    "p":  [[2, 1, 0, 13, 0.4248, 0.1962, 88, 11.2],
           [5, 3, 1, 17, 0.8200, 0.6004, 180, 7.9]],
    "dp": [[2, 0.700, 0.304, 0]],
    "ob": [[12, 2, 0], [41, 3, 1], [20, 1, 0]],
    "eq": [1, 0.52, 0.48, 7, 840]
  } }
```

Key	Meaning
`ph`	Game phase: 0 lobby, 1 countdown, 2 running, 3 paused, 4 finished
`tl`	Time left in seconds
`p`	Players: `[id, team 0–3, tone 0 light / 1 dark, flags, u, v, heading°, speed m/s]`; flags bit 0 active, 1 fallen, 2 carrying, 3 bot, 4 "Find me" active
`dp`	Delivery points: `[player id, u, v, state 0 active / 1 just delivered]`
`ob`	Obstacles: `[segment id 0–59, type 1 heavy traffic / 2 road work / 3 accident / 4 puddle, lane 0 = whole road]`
`eq`	Earthquake: `[active, centre u, centre v, magnitude 1–10, ms since start]`

The largest possible STATE (8 players, 8 delivery points, all 60 roads blocked) is about 750 bytes, well under the 1,400-byte limit for one Wi-Fi packet without fragmentation. "Find me" and the earthquake are part of the state, not one-time events, so a lost packet only delays them by 33 ms.

4.3 Other messages

```
{ "t": "C", "v": 1, "seq": 4, "ts": 0, "ack": true,
  "d": { "plan": "1.0.0", "bright": 0.6, "max_ma": 4000,
         "tones": [[80,160,255],[0,60,220],[90,255,90],[0,150,40],
                   [200,110,255],[110,0,200],[255,230,80],[210,140,0]],
         "anim": { "tail": 2, "dp_blink_hz": 1.0 } } }

{ "t": "V", "v": 1, "seq": 9, "ts": 61200, "ack": true,
  "d": { "mask": 5, "duty": 0.8, "ms": 1500, "ramp_ms": 200 } }

{ "t": "Q", "v": 1, "seq": 77, "ts": 123456, "ack": true,
  "d": { "id": 12, "src": "hit", "peak_g": 3.8, "mag": 6.1,
         "u": 0.31, "v": 0.72, "zone": 6, "conf": 0.7 } }

{ "t": "N", "v": 1, "seq": 9001, "ts": 123400,
  "d": { "hz": 100, "a": [0.01,0.02,0.01,0.85,2.90,3.80,1.20,0.40,0.10,0.03]
} }
```

```
{ "t": "T", "v": 1, "seq": 310, "ts": 310020,
  "d": { "fw": "0.4.1", "plan": "1.0.0", "last_seq": 52031, "fps": 60,
         "rssi": -52, "ma_est": 1850, "heap": 182000, "err": [] } }

{ "t": "O", "v": 1, "seq": 620, "ts": 310050,
  "d": { "ping_ts": 60790, "state_seq": 52031, "state_age_ms": 9 } }
```

- `V.mask` is a bit mask of the four corner motors (5 = north-west and south-east). `duty` 0–1, `ms` duration, `ramp_ms` soft start.
- `Q.src` is `hit` (hand) or `motor` (caused by our own motors). `zone` is the 3 × 3 area index (0 = north-west, 8 = south-east); `conf` is 0–1. `mag` is the measured strength on a 0–10 scale; M1 multiplies it by the admin strength setting.
- `N.a` is the acceleration size in g after removing gravity, used for the seismic graph.
- Board delay = half the ping round trip + `state_age_ms` (time from receiving a STATE to showing it on the LEDs).

4.4 Connection rules

1. At boot the board sends `HELLO` as a UDP broadcast every 1 s. The server answers to the board's address, and both sides then use direct messages. A fixed server IP can also be stored in the board's flash.
2. A message with `ack: true` is resent every 100 ms, up to 5 times, until an `A` arrives. The receiver ignores repeated sequence numbers.
3. If no STATE arrives for 2 s, the board shows the "searching" animation and goes back to step 1. Recovery after a router or server restart must take under 5 s.
4. The admin's added delay and packet loss for the board link are applied on the server side by M2, in both directions. The board firmware also has the same option for standalone tests.

4.5 Shared test vectors

A file `board_protocol_vectors.json` lists every example above with its exact MessagePack bytes in hex. The ESP32 code, the server code (M2) and both PC tools must encode and decode every vector correctly before integration starts.

5. Equipment and technology choices

Recommendation: an ESP32-WROOM-32E development board, Arduino-ESP32 3.x on PlatformIO, UDP with MessagePack, and a 6-axis IMU chosen between two candidates by a one-week bench test. Every choice favours parts and libraries that many people already use, because a rare bug in a rare tool can stop the whole board.

5.1 Microcontroller

Option	Strengths	Weaknesses	Verdict
ESP32-WROOM-32E DevKitC	Two cores (Wi-Fi on one, LEDs on the other); 8 RMT channels for LED data; largest library support; cheap and easy to find	No native USB (uses a USB–serial chip, which is fine)	Recommended
ESP32-S3 DevKitC-1	Two cores, more RAM, native USB	Only 4 RMT transmit channels; some LED libraries had S3-specific bugs	Good alternative
ESP32-C3 / C6	Cheap, modern	One core: Wi-Fi interrupts disturb LED timing and cause flicker	Rejected

We buy 3 boards: one in the LED board, two for development desks (one of them is the spare).

5.2 Firmware framework

Option	Strengths	Weaknesses	Verdict
Arduino-ESP32 3.x on PlatformIO	Fast to write; FreeRTOS tasks and core pinning still available; built-in unit tests and a native PC build	Arduino layer hides some settings (we set Wi-Fi power save off ourselves)	Recommended
ESP-IDF 5 (native)	Full control, official framework	Steeper learning curve; fewer ready LED and sensor drivers	Fallback for single parts only
MicroPython	Very easy	Too slow and jittery for 60 fps LED output and 1 kHz sensor reading	Rejected

5.3 Transport

Option	Strengths	Weaknesses	Verdict
UDP	Lowest delay; a lost packet never blocks the next one; admin packet loss is easy to simulate	We add our own acknowledgement for the few important messages (Section 4.4)	Recommended
WebSocket (TCP)	Reliable and ordered; easy to inspect in a browser	One lost packet delays every later packet (head-of-line blocking): visible stutter on the LEDs; reconnect logic needed	Rejected for state; possible for a debug console
MQTT	Common in IoT; topics	Needs a broker program, one more part that can fail and one more hop of delay	Rejected
ESP-NOW	Very low delay	Works only between ESP chips, not with the PC	Rejected

5.4 Message encoding

Option	Strengths	Weaknesses	Verdict
MessagePack	Compact; new fields can be added without breaking old code; read by ArduinoJson on the ESP32 and by standard libraries in Node.js and Python; easy to print as JSON	Slightly larger than a hand-made binary format	Recommended
JSON	Readable as text	2–3 times larger; the worst-case STATE could exceed one Wi-Fi packet	Debug mode only
Hand-made binary structs	Smallest	Every change breaks all sides; byte order and alignment bugs	Rejected

Option	Strengths	Weaknesses	Verdict
Protocol Buffers (nanopb)	Compact, typed	Code generation step on three platforms; more setup for little gain	Rejected

5.5 Motion sensor (IMU)

The proposal requires a gyroscope and an accelerometer. Two 6-axis candidates are bought and tested in week 1; the driver hides the difference from the rest of the firmware.

Option	Strengths	Weaknesses	Verdict
MPU-6050 (GY-521 module)	Very common, well documented, ±16 g and ±2000 °/s, built-in FIFO, very cheap	Old part, many clone chips of mixed quality	Candidate A
LSM6DS3TR-C module	Lower noise, larger FIFO, current production part	Fewer tutorials	Candidate B
ICM-42688-P	Best noise and speed	Harder to find locally; more complex driver	Only if A and B fail
4 piezo discs at the corners (extra)	Very cheap; strength ratio between corners gives a clear hit area	Needs 4 ADC inputs and wiring	Optional add-on if localisation accuracy is not reached

5.6 Parts list for M7

Part	Qty	Note
ESP32-WROOM-32E DevKitC	3	1 in the board, 2 for development and spare
MPU-6050 module	2	Candidate A
LSM6DS3TR-C module	2	Candidate B
Vibration motor and driver set	1	Details in Section 8
2.4 GHz Wi-Fi router, dedicated to the game	1	Same router for phones and board; no internet traffic on it
USB logic analyser, 8 channels	1	Shared with M8, for timing checks

5.7 Libraries and tools

ArduinoJson 7 (JSON and MessagePack), the built-in `WiFiUDP`, `Preferences` (settings in flash), `ArduinoOTA` (updates over Wi-Fi after the board is closed), the PlatformIO Unity test framework, and Python 3 with `msgpack` and `matplotlib` for the PC tools.

6. Firmware architecture

The firmware splits the two ESP32 cores by job: core 0 runs Wi-Fi, the network tasks, the sensor and the motors; core 1 runs only the LED engine (M8). This keeps Wi-Fi interrupts away from LED timing, the most common cause of flicker on ESP32 LED projects.

6.1 Tasks

Task	Core	Priority	Period	Job
`net_rx`	0	High	On each packet	Decode, check version and sequence, write state into the double buffer, send acks
`net_tx`	0	Medium	Queue driven	Send STATUS (1 Hz), SENSOR (10 Hz), PONG, resend unacknowledged QUAKE
`imu`	0	High	10 ms	Read 10 samples from the sensor FIFO (1 kHz), filter, detect hits, estimate area
`motor`	0	Low	10 ms	Soft start, duration and stop of vibration commands

Task	Core	Priority	Period	Job
`render` (M8)	1	Highest	16.7 ms (60 fps)	Build the LED frame and start the output
Task watchdog	—	—	3 s	Restarts the chip if any task hangs; the reason is reported in the next STATUS

6.2 Data hand-over between cores

The network task writes each new state into the free half of a double buffer and then switches one atomic index. The render task always reads the newest complete buffer. No lock is held while LEDs are being written, so a slow packet can never stop a frame and a frame can never stop a packet.

6.3 Timing budget per LED frame (16.7 ms)

Step	Budget
Copy newest state	< 0.1 ms
Build the frame (M8 animation engine)	⩽ 6 ms
LED output: 5 channels in parallel, longest 93 LEDs × 30 µs	about 2.8 ms, done by DMA hardware in the background
Free time	⩾ 7 ms

6.4 Start-up and failure handling

1. LEDs on with the boot animation (proves power and data lines).
2. IMU start and 2 s bias measurement; the board must be still. If it moves, measure again.
3. Wi-Fi connect with power saving off (power saving adds 100 ms or more of delay) and `HELLO` broadcast.
4. Running: STATE in, reports out.

If the IMU does not answer, the board keeps running without hit detection and reports `imu_missing` in STATUS. A brown-out or watchdog restart is reported the same way. Settings are kept in flash and can be changed over USB with a small text command set (`wifi`, `server`, `cal imu`, `cal zone`, `demo`, `stats`), so no re-flashing is needed to change networks.

7. Hit detection and earthquake localisation

One IMU under the centre of the board gives both the strength and the approximate area of a hit: the accelerometer measures how hard the board was hit, and the gyroscope measures which way it tilted. We commit to a 3 × 3 area (9 zones) with a confidence value, not to an exact point.

7.1 Detection

1. The sensor runs at 1 kHz (accelerometer ±16 g, gyroscope ±2000 °/s). The firmware reads its FIFO every 10 ms, so no sample is lost while Wi-Fi is busy.
2. Gravity and slow drift are removed with a high-pass filter (about 5 Hz). The result is the shake signal `a_hp`.
3. A hit starts when `|a_hp|` passes a threshold. The threshold adapts: the larger of a fixed minimum and 8 times the noise level of the last 5 s.
4. The firmware records a 150 ms window from the start, computes strength and area, and sends one `QUAKE` message.
5. A 1.5 s dead time follows, so one hit makes exactly one earthquake. If our own motors are running, the event is marked `src: motor`.

7.2 Strength

The magnitude uses the energy of the shake in the window, on a log scale like a real earthquake scale:

$$E = \int_{0}^{150\,ms} |a_{hp}(t)|^2\,dt, \qquad mag = \mathrm{clamp}\left(k \cdot \log_{10}\frac{E}{E_0},\ 0,\ 10\right)$$

`E0` and `k` come from the drop-test calibration (Section 9). M1 multiplies `mag` by the admin strength setting, so the admin can still change the earthquake size at any time.

7.3 Area

A downward hit at point (x, y) from the board centre turns the board around its feet. The turning (torque) is the hit force times the distance, so the first gyroscope reading points to the side of the hit, and the ratio of turning to force gives the distance:

$$\tau = \vec{r} \times \vec{F} \;\Rightarrow\; \hat{x} \propto \frac{\Delta\omega_y}{\Delta a_z}, \quad \hat{y} \propto -\frac{\Delta\omega_x}{\Delta a_z}$$

In practice the board, feet and table are not ideal, so the firmware does not use fixed constants. It uses a small linear model (2 outputs, 4 inputs) whose coefficients are fitted from calibration taps on known points and stored in flash. The output is `u, v`, the zone (0–8) and a confidence from the signal size and the fit error.

7.4 Targets and fall-back

Measure	Target
Detection of a standard calibrated hit	⩾ 95%
False earthquakes in normal play (leaning, putting a phone down)	⩽ 1 per 30 minutes
Correct zone (3 × 3)	⩾ 80%
Correct quarter of the board	⩾ 95%
Hit to QUAKE message sent	⩽ 200 ms

This needs two mechanical conditions from M8: the board stands on four soft rubber feet so it can tilt slightly, and the IMU is screwed (not glued) at the centre with its axes along the roads. If the zone target is not met by the end of week 4, we add the optional four corner piezo discs from Section 5.5. If confidence is low during a game, the message still carries the best estimate and M1 may use the nearest quarter.

8. Vibration motor control

Four small DC motors with 3D-printed off-centre weights sit under the four corners of the board. Running one or two of them shakes that side of the board, so the admin can start an earthquake at a chosen place and the IMU detects it through the same path as a hand hit.

8.1 Hardware

Part	Choice	Reason
Motors	4 × 130-size DC motor, 3–6 V, with a printed eccentric weight (+2 spares)	Strong enough to move a 60 × 60 cm board; cheap; easy to mount
Driver	Logic-level N-MOSFET per motor (for example AO3400 or IRLZ44N module), 100 Ω gate resistor, 10 kΩ pull-down	Switches 1–2 A directly from a 3.3 V pin; motor is off if the ESP32 resets

Part	Choice	Reason
Protection	Schottky diode (1N5819 or SS34) and 100 nF capacitor across each motor; 3 A fuse	Stops voltage spikes and radio noise from resetting the ESP32
Power	Separate 5 V, 3 A supply for motors; common ground with the ESP32	Motor start current must not dip the LED or ESP32 supply
PWM	ESP32 LEDC at 20 kHz, 10-bit	Above hearing range, smooth strength control

Alternative	Why not chosen
Coin vibration motors (phone type)	Too weak to shake a 60 × 60 cm board
One large motor in the centre	Cannot choose the area of the earthquake
Solenoid that knocks the board	Very realistic "hit", but loud, needs 12 V and a stronger driver; kept as a backup idea
L298N driver board	Loses about 2 V and gets hot

8.2 Control rules

1. A `VIBRATE` message sets the motor mask, strength (duty 0–1), duration and soft-start time. The admin's earthquake place is turned into the nearest one or two corner motors by M1.
2. Strength is mapped to duty through the calibration table (Section 9), so "magnitude 5" gives about the same shake on every board build.
3. Safety limits are in firmware, not on the server: at most 5 s on per command, at least 5 s rest after a long run, and all motors stop if the server link is lost.

9. Calibration procedures

Five calibrations are done on the finished board, in this order. Each is started with a USB command, saves its result in flash and writes a CSV file for the report. They are repeated whenever the board is moved to a different table.

Step	What	How	Saved result	Accept when
C1	Sensor bias	Board still for 2 s at every boot (automatic)	Gravity direction, gyro offset	Gyro offset < 1 °/s after correction
C2	Noise floor	60 s of normal room activity, nobody touching	Noise level for the adaptive threshold	No false trigger in a following 30-minute test
C3	Strength scale	Drop a 50 g steel ball from 5, 10, 20 and 40 cm onto the centre, 5 times each	`E0` and `k` for the magnitude formula	Each height gives a clearly different magnitude; spread within one height ⩽ 1.0
C4	Area model	Tap the 9 zone centres 10 times each with the standard tap (ball from 20 cm), then 25 random points for checking	Linear model coefficients	Zone ⩾ 80% and quarter ⩾ 95% correct on the check points
C5	Motor strength	Each motor alone at duty 30–100% in 10% steps	Duty for magnitudes 1–10 per motor; detected zone per motor	Detected zone is the motor's own corner zone in ⩾ 9 of 10 runs

The falling ball makes a hit that anyone can repeat: the same ball from the same height gives the same energy. That is why it is used instead of a hand for calibration. Two network values are also checked once on the final router setup: the round-trip time between server and board (target median < 15 ms, 95th percentile < 40 ms) and the packet loss without admin impairment (target < 0.1%).

10. Standalone testing

M7 is tested with two PC tools that M7 also delivers to the other modules, so nobody waits for the real board or the real server.

Tool	Replaces	What it does
`server_emu.py`	M1 and M2	Sends STATE at 10–60 Hz from scenario files, with adjustable delay and packet loss; shows QUAKE, SENSOR and STATUS live; plots the seismic signal

Tool	Replaces	What it does
`board_emu.py`	The LED board	Listens like the board, draws the city and LEDs in a window, answers HELLO and PING; lets M1 and M2 test before the board exists
Native test build	The ESP32	Runs the protocol code and hit-detection maths on the PC with recorded sensor data

Test cases

ID	Test	Method	Pass criterion
T1	Protocol vectors	Encode and decode every example in the shared vector file	All match byte for byte
T2	Bad packets	Random bytes, wrong version, old sequence numbers	Ignored and counted in STATUS; no crash
T3	Steady link	2 hours of STATE at 30 Hz	No restart; ≥ 99.9% of packets applied
T4	Delay	PING/PONG over the dedicated router	Median < 15 ms, 95th percentile < 40 ms
T5	Packet loss	20% loss set in the emulator	LEDs keep moving; acknowledged messages always arrive
T6	Recovery	Restart the router, then the server	Board back in sync within 5 s each time
T7	Hit detection	Recorded hit files on the PC, then live taps	Targets of Section 7.4 met
T8	False triggers	30 minutes of normal handling around the board	≤ 1 false earthquake
T9	Motors	Each motor, each strength; then link loss during a run	Correct motor runs; all stop within 2 s of link loss
T10	Load	LEDs at 60 fps, sensor at 1 kHz and STATE at 60 Hz together	No missed sensor samples; frame time jitter < 2 ms
T11	Update	Firmware update over Wi-Fi with the board closed	Works 5 times in a row

Sensor data from every live test is saved, so detection changes can be checked again on the PC without hitting the board.

11. Integration with other modules

M7 sits between the server modules and the board hardware, so it is integrated from both sides: first each side alone with the emulators, then together.

Order	Module	What is exchanged	Integration test	Pass criterion
1	M8 LED engine	`BoardState`, `BoardConfig`, link status (C++ calls)	`server_emu.py` scenario drives the real LEDs	Every scenario frame looks the same on the board and in `board_emu.py`
2	M2 Network	STATE, PING/PONG, admin delay and loss	Real server sends to the board; admin sets 100 ms delay, then 10% loss	Board delay graph shows +100 ms; LEDs keep moving at 10% loss
3	M1 Game Server	STATE content; QUAKE in; VIBRATE out	Alignment scenario: a player placed on every intersection in turn	LED dot on the right intersection every time; same place in the 3D view
4	M1 earthquake	QUAKE from a hand hit and from motors	Hit each zone; start a motor earthquake from the panel	Earthquake in the game within 300 ms, near the hit area
5	M6 Admin Panel	SENSOR, STATUS, PONG	Live seismic graph and board delay graph during a game	Graphs update; values match `server_emu.py` readings

The version check is part of every connection: `HELLO` carries the firmware and city plan versions, and the server refuses to start a game if the city plan versions differ.

12. Risks, work plan and open decisions

Risk	Likelihood	Impact	Mitigation
Wi-Fi delay spikes in a crowded classroom	High	High	Dedicated router on a free channel; power saving off; state-based protocol tolerates loss
LED flicker caused by Wi-Fi	Medium	High	Two-core split, DMA-based LED output, test T10
Hit area not accurate enough with one IMU	Medium	Medium	Commit to 9 zones with confidence; optional piezo add-on decided at week 4
Motor noise resets the ESP32	Medium	High	Separate motor supply, diodes, capacitors, fuse
IMU clone chip faulty	Medium	Low	Two candidate sensors, two of each, driver abstraction
Board closed and firmware needs a fix	High	Medium	Over-the-air updates, test T11
Server not ready for board tests	Medium	Medium	`server_emu.py` covers all messages

Work plan

Week numbers count from the approval of this proposal.

1. **Weeks 1–2:** Protocol vectors, `server_emu.py`, `board_emu.py`, Wi-Fi and UDP on the ESP32, tests T1–T3. IMU bench test chooses candidate A or B.
2. **Weeks 3–4:** Hit detection on recorded data, then live; motor drivers on the bench; tests T4–T9. Piezo decision at the end of week 4.
3. **Weeks 5–6:** Integration step 1 with M8 on the real board; calibrations C1–C5; test T10.
4. **Weeks 7–8:** Integration steps 2–5 with M1, M2 and M6; over-the-air update; test T11.
5. **Weeks 9–10:** Long tests, final calibration on the demo table, report data.

Decisions needed from the group before submission

- [ ] UDP with MessagePack for the board link (M2 must implement the same encoder)
- [ ] Where admin delay and loss are applied for the board link (M7 proposes M2, on the server)
- [ ] Dedicated Wi-Fi router for the demo (needed by M2 and M5 too)

- [ ] Earthquake strength rule: M1 multiplies measured `mag` by the admin setting
- [ ] Pin map in Section 3.3 agreed with M8